MEMORIA TÉCNICA Y DE DESARROLLO

Videojuego de Consola en Java: "Piedra, Papel o Tijera (JuegoPPT)"

Módulo:0485 Programación

Curso:1.º Formación Profesional DAW (Desarrollo de Aplicaciones Web)

Evaluación:Primera Evaluación — Entrega 1 (Versión v1.0)

DATOS DE IDENTIFICACIÓN

- Alumna: Danna Medilef Carrera Rodriguez
- Grupo / Modalidad: 1.º DAW — Individual
- Título del Videojuego: JuegoPPT (Piedra, Papel o Tijera)
- Curso Académico:2026/2027
- Repositorio de GitHub: https://github.com/Medi-19931/JuegoPPT.git

1. DESCRIPCIÓN GENERAL Y REGLAS DEL JUEGO

1.1. Objetivo y Temática

"JuegoPPT" es un videojuego interactivo de consola programado en Java que reinterpreta el clásico juego de "Piedra, Papel o Tijera". Permite realizar enfrentamientos estratégicos en duelos cerrados , máximo de 3 rondas (al mejor de 3 asaltos). El software incorpora dos modalidades principales (contra una inteligencia artificial o en multijugador) y un módulo analítico avanzado de estadísticas por sesión.

1.2. Reglas Mecánicas del Juego

1. Estructura de Partida: Toda partida (Solitario o Multijugador) se hace como máximo 3 rondas (MAXRONDAS = 3).

2. Opciones Disponibles: En cada turno, el jugador debe seleccionar numéricamente su elección:

- 1: Piedra
- 2: Papel
- 3: Tijera

3. Matriz de Combate:

- La Piedra (1) vence a la Tijera (3) y pierde contra el Papel (2).
- El Papel (2) vence a la Piedra (1) y pierde contra la Tijera (3).
- La Tijera (3) vence al Papel (2) y pierde contra la Piedra (1).
- Opciones iguales resultan un Empate (0).

4. Condición de Victoria / Definitiva: Se acumulan las victorias parciales de cada ronda. Al finalizar las 3 rondas, el programa declara al ganador definitivo de la partida comparando los marcadores parciales.

2. FUNCIONALIDADES IMPLEMENTADAS Y DIFERENCIAS RESPECTO A LA PROPUESTA INICIAL

2.1. Funcionalidades del Menú Interactivo

El sistema se controla mediante un menú interactivo continuo basado en un bucle “do-while” que valida las entradas del usuario (opciones del 1 al 5):

1. Opción 1 - Modo Solitario (Jugador 1 vs Consola): El Jugador 1 enfrenta a la IA del sistema. La consola genera sus jugadas aleatoriamente mediante “Math.random()”.

2. Opción 2 - Modo Multijugador (Jugador 1 vs Jugador 2): Duelo local por consola donde dos jugadores introducen de manera consecutiva sus elecciones por turno.

3. Opción 3 - Ver Estadísticas de la Sesión: Pantalla analítica global e histórica. Muestra el número total de partidas jugadas, el desglose por modalidad, las victorias acumuladas por cada bando y el historial ronda por ronda de la última partida jugada utilizando vectores.

4. Opción 4 - Ver Reglas e Instrucciones: Muestra en pantalla el manual de usuario, explicaciones de como jugar y el rango de entradas válidas.

5. Opción 5 - Salir del Juego: Cierra la aplicación de manera controlada finalizando el bucle del menú y liberando los recursos de consola (scanner.close()).

2.2. Evolución sobre la Propuesta Inicial

En la propuesta inicial se planeó un sistema básico de comparación de elecciones. Durante la fase de implementación se han añadido mejoras significativas:

- Persistencia de sesión: Acumuladores globales que mantienen el conteo de partidas y victorias a lo largo de múltiples ejecuciones del menú sin reiniciar contadores.

- Formateo avanzado de salida: Empleo de “System.out.printf()” para alinear perfectamente la tabla de marcadores parciales.

- Uso del historial de rondas: El vector “historialRondas” almacena los resultados de cada asalto (0 para empate,1 para el rival y 2 para el Jugador 1). Mediante la variable modoJuego, el sistema traduce el código 1 a "Consola" o "Jugador 2" según corresponda al mostrar las estadísticas.

3. DISEÑO TÉCNICO Y ARQUITECTURA DEL CÓDIGO

El software ha sido diseñado aplicando las buenas prácticas de programación en Java, evitando el uso de librerías complejas no evaluadas en la primera evaluación.

3.1. Identificadores, Variables y Constantes

Constantes:

final int MAXRONDAS = 3; (Define el límite inmutable de rondas por partida).

Contadores de Sesión (Variables Globale/Acumuladores):

int partidasTotales, int partidasSolitario, int partidasMultijugador, int empatesTotales.

int victoriasGlobalesJ1, int victoriasGlobalesConsola, int victoriasGlobalesJ2.

Variables de Control e Interfaz:

int opcionMenu (Almacena la navegación del menú principal).

int modoJuego (Identifica el modo activo: 1 Solitario, 2 Multijugador).

String resultadoFinal (Cadena que almacena la etiqueta del ganador global).

Variables Parciales por Partida:

int victoriasJ1, int victoriasConsola, int victoriasJ2, int opcionJ1, int opcionRival.

3.2. Estructuras de Control Utilizadas

- Bucle Repetitivo do-while: Mantiene el programa en ejecución activa mostrando el menú principal hasta que opcionMenu == 5.

- Sentencia Condicional if - else if - else: Utilizada en múltiples niveles:

1. Selección de opción del menú principal.

2. Control y validación de entrada de datos (comprobación de rango 1 <= opcion <= 3).

3. Lógica de comparación de reglas de juego (combate Piedra/Papel/Tijera).

4. Determinación del ganador definitivo tras completar el bucle de rondas.

- Bucle Determinado for:

1. Recorrido de las 3 rondas obligatorias por partida (for (int i = 0; i < MAXRONDAS; i++)).

2. Despliegue dinámico de las opciones de jugada desde el array opciones.

3. Recorrido analítico del vector historialRondas en el módulo de estadísticas.

3.3. Uso de Arrays Unidimensionales (Vectores)

El programa utiliza dos arrays unidimensionales con funciones fundamentales:

1. String[] opciones = {"Piedra", "Papel", "Tijera"}; : Vector de lectura usado para representar los valores numéricos ingresados con su representación textual en pantalla.

2. int[] historialRondas = new int[3]; : Vector de almacenamiento de tamaño 3.

- Función: Registra el resultado numérico de cada ronda individual durante la partida (0 = Empate, 2 = Victoria Jugador 1, 1 = Victoria Consola/Jugador 2).

- Procesamiento: En la Opción 3 de estadísticas, este vector se recorre con un bucle `for` para traducir cada código numérico a una descripción textual entendible para el usuario según la variable `modoJuego`

4. PRUEBAS REALIZADAS Y VALIDACIÓN (TESTING)

Para garantizar la estabilidad del software, se ejecutaron diversas baterías de pruebas:

4.1. Pruebas de Casos Normales

Entrada: Selección Opción 1 -> Elegir 1 (Piedra) en Ronda 1, 2 (Papel) en Ronda 2, 3 (Tijera) en Ronda 3.

Resultado Esperado: La consola responde generando su elección, determina el ganador de cada ronda, guarda el estado en historialRondas, muestra la tabla formateada y declara el ganador final.

Resultado Obtenido: Correcto.

4.2. Pruebas de Entradas Incorrectas y Casos Límite

Entrada Menú: Introducir un número fuera de rango como 8 o -1.

Resultado Esperado: El programa muestra un mensaje de advertencia ("INGRESE SOLO UN NUMERO DEL 1-5...") y vuelve a desplegar el menú sin bloquearse.

Resultado Obtenido: Correcto.

Entrada Jugada: Introducir `9` al seleccionar Piedra/Papel/Tijera.

- Incidencia detectada en la prueba de jugadas inválidas: Si el usuario introduce un número fuera de rango (9), se muestra el aviso de error por consola, pero la ronda se salta sin acumular correctamente en el historial o en los marcadores, dejando los resultados a cero (empate técnico implícito). Queda pendiente para una próxima versión validar que la ronda se repita o se gestione con un bucle específico de validación de entrada.

5. DIFICULTADES ENCONTRADAS Y POSIBLES AMPLIACIONES

5.1. Dificultades y Soluciones

1. Sincronización de estadísticas globales vs. locales: Inicialmente, los contadores de la partida actual interferían con los acumuladores de la sesión.

Solución: Se separaron las variables en dos bloques explícitos: contadores parciales que se reinician al iniciar cada partida (victoriasJ1 = 0) y acumuladores globales que conservan su valor durante todo el ciclo del do-while.

2. Interpretación del Array de Historial: Distinguir si la posición 1 del vector correspondía a la Consola o al Jugador 2.

Solución: Se integró la variable modoJuego dentro de la lectura del array en la opción de estadísticas para adaptar la salida textual.

5.2. Propuestas de Ampliación para la Segunda Evaluación

Mejorar la validación de las entradas del usuario para que el programa controle mejor si se introduce un número incorrecto.

Añadir nuevos mensajes o contadores en la pantalla de estadísticas para hacerla más visual.

6. INSTRUCCIONES DE COMPILACIÓN Y EJECUCIÓN

Requisitos previos:

- Tener instalado el JDK de Java (versión 17 o superior).
- Entorno de desarrollo (Visual Studio Code o Símbolo del sistema de Windows).

Pasos para compilar y ejecutar desde la terminal (CMD):

1. **Abrir la terminal y situarse en la carpeta raíz del proyecto:** Abre el Símbolo del sistema y navega hasta la carpeta principal del juego (JuegoPPT):

ejemplo:

```
None
    cd C:\Users\DELL\Documents\JuegoJava\JuegoPPT
```

2. **Compilar el código fuente:** Ejecuta el siguiente comando para compilar la clase `App.java` que está dentro de la carpeta `src` y guardar el ejecutable compilado en la carpeta `bin`:

ejemplo:

```
None
      javac src/App.java -d bin
```

3. **Ejecutar el juego:** Una vez compilado sin errores, inicia la aplicación en la consola con el comando:

ejemplo:

```
None
      java -cp bin App
```

*(Nota: En Visual Studio Code o Eclipse también se puede ejecutar directamente pulsando el botón de **Run / Debug** sobre el archivo* `App.java`*).*